



CODING STANDARDS

Software Engineering II - Workshop #1

Group Members:

Michael Crescensio Poveda Quimiz
Bruno Rafael Romero Yagual

Table of Contents

1.	Introduction	2
2.	Development and Refactoring Process	3
2.1.	Project Repository.....	3
2.2.	Initial Diagnosis and Quality Baseline (reports/initial.html)	3
2.3.	Mitigation Strategy via Incremental Commits.....	4
2.4.	Rule Suppressions and Technical Justifications	5
2.5.	Final Quality State (reports/final.html)	6
3.	Conclusions.....	7
4.	Recommendations	7

1. Introduction

Maintaining a large-scale software system depends heavily on the readability, consistency, and alignment of its source code with industry-standard design conventions. Adhering to these engineering practices radically lowers the cognitive load of engineering teams, mitigates the introduction of subtle runtime regressions, and fosters predictable collaborative workflows. However, human-led code reviews are often slow, subjective, and susceptible to oversights. To address this, software engineering relies on automated static analysis tools, commonly known as linters, which programmatically analyze codebase structure, architectural hygiene, and style compliance without requiring code execution.

The primary objective of this project was to refactor a legacy, intentionally flawed financial module responsible for evaluating credit eligibility and loan terms within a credit union (cooperativa de ahorro y crédito). Using Pylint as the automated quality gate within a Visual Studio Code environment, code smells and structural defects inside the target module (`src/loan/eligibility.py`) were systematically diagnosed, isolated, and remediated. This refactoring operation was executed under a strict technical constraint: eliminate all architectural and stylistic violations reported by the linter while preserving the core underlying business rules and ensuring that the pre-existing test suite remained completely operational and green.

2. Development and Refactoring Process

2.1. Project Repository

<https://github.com/mcpoveda2/is2-coding-standards-g8-python>

2.2. Initial Diagnosis and Quality Baseline (reports/initial.html)

Before making any structural changes, a comprehensive static analysis scan was triggered across the repository to establish a quantitative baseline. The resulting diagnostic report, preserved in `reports/initial.html`, flagged a total of 40 quality violations confined entirely within `src/loan/eligibility.py`. The structural and stylistic defects were classified across the following recurring technical categories:

- **Deep Conditional Nesting (Pyramid of Doom):** The primary evaluation routine contained highly convoluted if-else blocks to validate members' credit criteria (income, age, debt-to-income ratio, and job stability). This anti-pattern severely inflated the module's cyclomatic complexity, triggering too-many-branches and too-many-statements alerts.
- **Mutable Default Arguments:** The core function declaration used a mutable list as a default parameter value (`history=[]`). In Python, this pattern forces the default object instance to be shared across all consecutive function calls, causing severe state leakage and unpredictable side effects.
- **Stylistic and Documentation Omissions:** The legacy module suffered from a total lack of documentation strings (missing-function-docstring, missing-module-docstring), excessive line lengths exceeding standard specifications (line-too-long), unnecessary parentheses (superfluous-parens), and redundant boolean checks (e.g., comparing `is_pensioner == True`).
- **Dead Code and Unused Variables:** Orphaned variables such as `tmp = 0` were declared and initialized within the execution path but never read, contributing to code bloating and violating the unused-variable rule.

2.3. Mitigation Strategy via Incremental Commits

The technical remediation process was conducted iteratively using small, atomic changes. Each specific modification was committed independently to the version control system, explicitly referencing the precise Pylint alphanumeric code or standard rule name within the commit message. This maintained granular history and guaranteed full traceability of the code quality improvement process. Key examples of these standardized commits include:

- **pylint(W0612): remove unused variable tmp** -> Deletion of dead intermediate variables from the primary evaluation execution path.
- **pylint(C0116): add missing function docstrings to newly public financial validation and terms algorithms** -> Injecting robust inline documentation and interface definitions compliant with PEP 257.
- **pylint(C0303): strip trailing whitespace from terms calculation and loan evaluation steps** -> Purging hidden layout issues and formatting the source text cleanly.
- **pylint(R0913, R0917, R0911, R0914): apply tactical pylint suppression comments for compliance-enforced function signatures** -> Injecting scoped ignore tags to safeguard mandatory parameter interfaces requested by corporate integration layers.
- **pylint(python:S2589): simplify status_tag check and remove redundant condition** -> Refactoring logic expressions to strip away tautologies and redundant runtime validations.
- **pylint(python:S125): remove commented-out code** -> Discarding blocks of legacy dead code preserved inside comments, ensuring a clean maintainable module.

To structurally resolve the 'Pyramid of Doom' without altering the credit union's internal policy logic, the sprawling monolithic function was broken down into specialized, isolated modules. Complex logical sub-routines were extracted into distinct helper functions: `check_credit_eligibility()`, `calculate_late_payments_score()`, `calculate_employee_pensioner_terms()`, and `calculate_loan_terms()`. This separation of concerns enabled the application of guard clauses, allowing the functions to exit early whenever a policy precondition failed. This design flattened the control flow, dramatically simplified code navigation, and permanently resolved the cyclomatic complexity warnings.

2.4. Rule Suppressions and Technical Justifications

Per strict rigorous engineering guidelines, suppressing static analysis warnings must never be used to hide poor design. However, select suppressions were tactically introduced using localized inline comments (# pylint: disable=...) where modifying the signature would break external API contracts, breach strict banking regulatory bounds, or cause scope creep within the testing pipeline:

Pylint Rule ID	Target Context	Technical Justification
too-many-arguments too-many-positional-arguments	check_credit_eligibility calculate_loan_terms evaluate	The financial risk domain inherently demands checking multiple parallel metrics simultaneously (income, existing debt, age, cooperative seniority, dependents, guarantor flags). Packing these primitive types into dynamic containers would obscure contract visibility and break binding integrations with pre-existing black-box unit tests.
too-many-return-statements	check_credit_eligibility	Implementing a clean, sequential validation flow via guard clauses naturally generates multiple deterministic exit points as credit criteria fail. Forcing a single exit point would require reintroducing artificial state flags, lowering readability and increasing cognitive complexity.
broad-exception-caught	calculate_loan_terms	A generic exception handler was implemented at the edge of the auxiliary calculation block to trap malformed inputs or downstream numerical overflows. This acts as an engineering safety shield, preventing complete process crashes during nightly automated batch updates.

2.5. Final Quality State (reports/final.html)

Following code refactoring, encapsulation of complex logic, and documentation of approved rule suppressions, the codebase was re-analyzed using Pylint. The updated automated quality report generated at `reports/final.html` officially verifies a clean status of zero (0) design or code quality violations within the modified files.

Crucially, to verify that the refactoring did not introduce functional regressions or alter complex business rules, the module was validated against the pre-existing system test suite. The entire test suite executed perfectly and remained 100% green, proving that the structural changes improved code quality and maintainability without altering the application's runtime behavior.

Pylint report

Report generated on 2026-27-05 at 20:41:52 using [pylint-report](#)

Score: 10.00 / 10

- Command line (0)
- Command line or configuration file (0)
- loan.eligibility (0)

3. Conclusions

- **Effectiveness of Automated Static Analysis:** Static analysis engines like Pylint are vital for modern software engineering. They offer an objective, automated metric to uncover architectural blind spots and quantify technical debt in legacy environments, providing actionable feedback that guides engineering refactoring phases.
- **Impact of Clean Code Architecture:** Replacing nested 'Pyramid of Doom' conditional loops with clear modular helpers and guard clauses drastically minimized the file's cognitive load. The final architecture is not only clean of linter alerts but is also highly maintainable, intuitive, and prepared for future changes to banking policy.
- **Navigating Legacy System Friction:** Interacting with unstandardized codebases naturally causes initial engineering friction due to structural fragility. However, this project confirms that systematic, test-driven, atomic refactoring eliminates integration risks and pays off immediately in team velocity and system stability.
- **Pragmatic Balance in Rules Enforcement:** Coding standards should be enforced thoughtfully, not rigidly. The documented use of tactical, inline suppressions proves that standard code constraints must adapt to real-world integration requirements, external testing constraints, and legacy API signatures.

4. Recommendations

- **Adopt Shift-Left Code Quality Tools:** Integrate automated linters directly into developers' local IDEs (e.g., VS Code Pylint extension) to trigger on-save. Catching syntax, style, and structural issues during active coding prevents the accumulation of technical debt and instills strong code habits across the team.
- **Enforce Quality Gates in CI/CD Pipelines:** Incorporate an automated validation workflow (such as GitHub Actions) into the central source repository. Configuring a pull request step that blocks merges if the linter error count exceeds zero guarantees that no code degradation can enter the main production branch.
- **Introduce Static Type Hinting:** Complement Pylint styling by adopting Python type hints (e.g., `income: float, is_employee: bool`) across the financial module. Merging type definitions with linting checks improves documentation and lets static type Checkers like Mypy block data type anomalies before runtime.
- **Calibrate Linter Configurations Over Time:** Establish periodic, bi-annual reviews of the team's linter rule configurations (e.g., `.pylintrc`). Adjusting rule limits (like maximum allowed positional arguments) ensures that code quality enforcement evolves smoothly alongside changing business complexity and team design patterns.